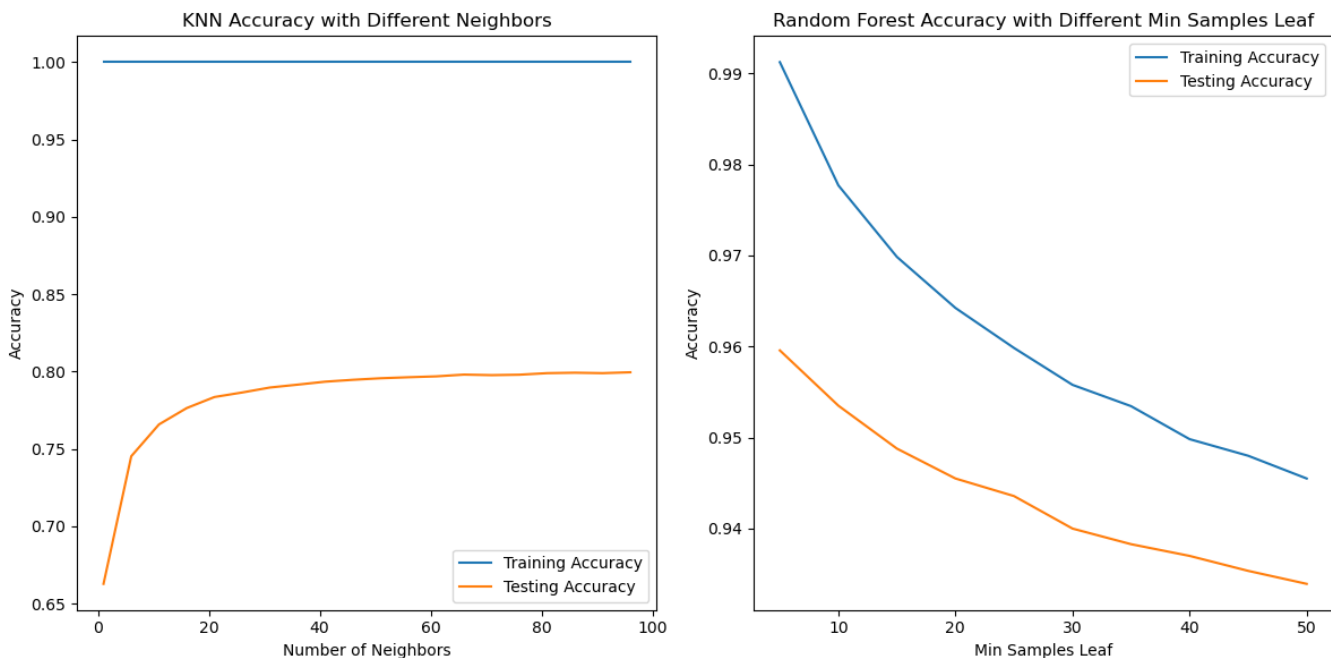


1. In the first scenario, we're working with noiseless input images and labels. The aim is to explore hyperparameter tuning for KNN classifiers and decision trees. For weighted KNN Classification we need to select the number of neighbors when a Euclidean distance measure. For Random Forests, our exploration involves finding the minimum number of samples required for a leaf node for up to 100 forests. During model training, we aim to find the hyper-parameters that deliver the best performance for both the training and test sets. With this context, using MNISTclassification1.py, please respond to the questions below.

a) For KNN with distance weighting, plot the accuracy of training and testing with respect to the number of neighbors. Vary the number of neighbors from 1 to 100 in steps of 5 (1, 6, 11, 16 ...). What is the best number of K? Justify your selection.

b) For Random Forests, plot training and test accuracy as a function of the minimum number of samples for a leaf node (from 5 to 50 with 5 steps). Select the best hyper-parameter, and justify your choice.

The result plot is shown as followed:



the best number of K for KNN is 96, because by this code:

```
# Find the index of the highest testing accuracy for KNN
best_index_KNN = np.argmax(R_test_KNN)
# Get the corresponding best number of neighbors
best_neighbour = Neighbours[best_index_KNN]
```

we find that the best number of neighbours for KNN: 96, with the highest testing accuracy for KNN: 0.7995

and the best number of samples for random forest is 5, because by this code:

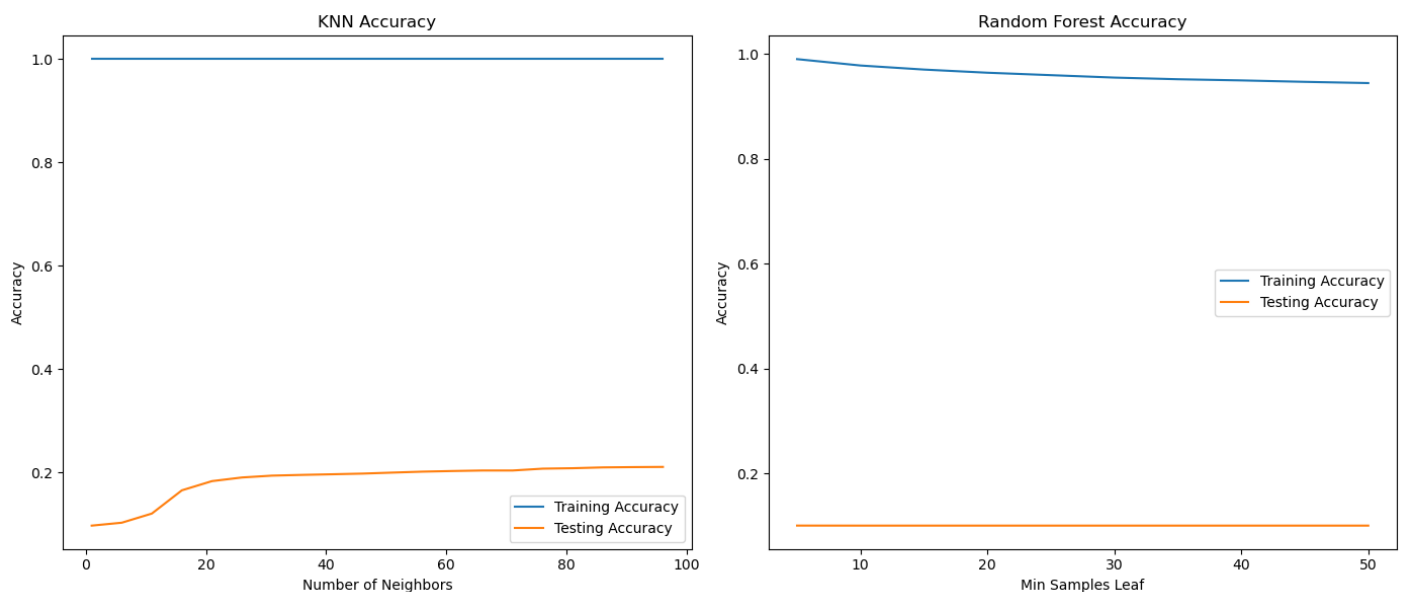
```
# Find the index of the highest testing accuracy for Random Forest
best_index_RF = np.argmax(R_test_RF)
# Get the corresponding best min_samples_leaf
best_min_samples_leaf = Min_sample[best_index_RF]
```

we find that the best min_samples_leaf for Random Forest: 5, with highest testing accuracy for Random Forest: 0.9586428571428571

2. In the second scenario, our objective is to delve into the concept of generalization. Typically, we expect a model trained on a large training set to exhibit similar performance on a smaller test set, assuming both are drawn from the same distribution. Practical experience often contradicts this assumption however. In reality, the test set distribution may not align perfectly with the distribution of training data, leading to challenges in obtaining good generalization. This underscores the importance of critically assessing model performance in real-world scenarios where data distributions may deviate from commonplace idealized assumptions. To mimic this behaviour, we add some noise to the test set. With this new data, please answer these questions, using MNISTclassification2.py.

a) Perform a hyper-parameter search for KNN and Random Forest and report the best hyperparameter for each method.

The plot for KNN and random forest:



Best K for KNN: 96 with Test Accuracy: 0.21314285714285713

Best Min Samples Leaf for Random Forest: 5 with Test Accuracy: 0.10264285714285715

b) Discuss the results based on both algorithms and how noise can increase or decrease performance.

In senario 1, the KNN produce a decent testing accuracy, but in senario 2, noise significantly

impacts the KNN algorithm's ability to classify images accurately and the accuracy decreases dramatically. The high value of k implies that the model needs to consider large amount of sets of neighbors to make a reliable prediction, probably because the noise makes the data more homogeneous, and thus, individual close neighbors are not as trustworthy.

Similary, random forest also show low testing accuracy, suggesting that the Random Forest model might be overfitting. At lower `min_samples_leaf` values, maybe allowing the trees to grow more complex (less pruning) does not lead to better generalization with the noisy test data.

The noise, in this case:

Increase in Overfitting: Both algorithms show a dramatic drop in test accuracy compared to training accuracy, indicating that the models are not generalizing well to the noisy data. This could be due to overfitting, where the models learn the noise as a feature, reducing their ability to predict unseen data accurately.

Decreased Model Robustness: The additional noise can mess up with the underlying patterns that the models are supposed to learn, leading to a decrease in performance. It is particularly impactful for KNN, which relies on the assumption that similar examples are near each other in the feature space. Random Forest, though have more resistancy, also suffers when the noise disrupts the data's inherent structure.

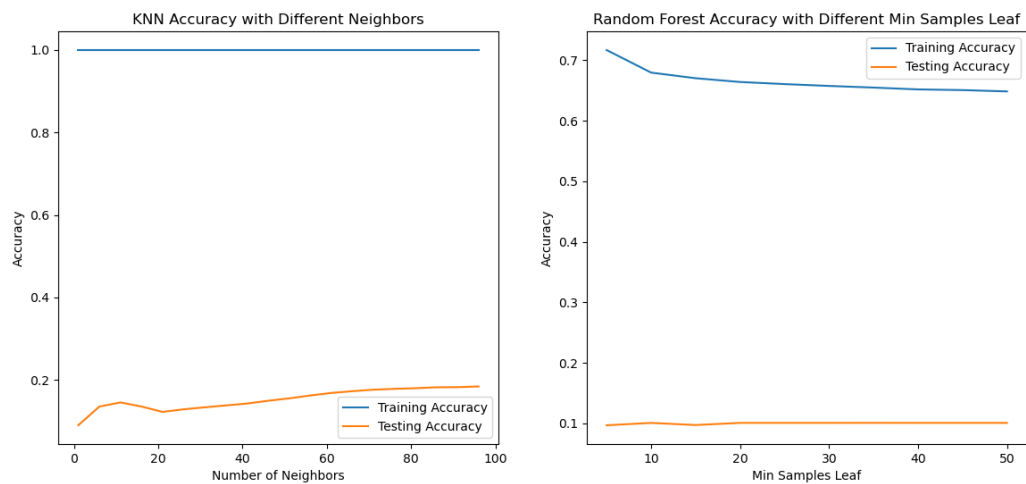
c) Propose a method to enhance overall performance. Note that there is no single correct answer.

We may preprocess the training data before sending them to the model, for example, we may apply outlier detection to prevents potential noise, filtering and reduce noise in the data. We may also apply are more complex distance weighting scheme that can reduce the influence of noisy instances.

3. In the third scenario, our objective is to assess the performance of the model in the context of dealing with noisy labels. Above we assumed noiseless labels, but practical experience shows that such noise is common in real-world datasets. This noise can stem from various sources, including human error in the annotation process, leading to less precise labeling. The introduction of such noise into the data has the potential to reduce a model's ability to generalize effectively, highlighting the need for robust strategies to handle and mitigate the impact of noise in the training and evaluation processes. To this end, we add noise to the training labels. Using the `MNISTclassification3.py`, answer the following questions:

a) Perform a hyper-parameter search for KNN and Random Forest and report the best performance achieved by each method.

The plot and the best parameter for KNN and random forest:



KNN Best Performance: $n_neighbors = 61$, Accuracy = 0.20842857142857144

Random Forest Best Performance: $min_samples_leaf = 25$, Accuracy = 0.10307142857142858

b) Discuss how the optimal parameters for KNN change between Scenario 1 above and Scenario 3 here.

In Scenario 1, with noiseless input images and labels, the optimal number of neighbors for KNN was 96 with a high testing accuracy of 0.7995. The larger number of neighbors ($k=96$) might smooth out the decision boundary and make the classification more robust to any small variations within the noiseless dataset.

However, in Scenario 3, once noise is introduced into the training labels, the optimal number of neighbors changes to 61 (which is lower than in Scenario 1) along with a dramatic reduce in accuracy to 0.2084, indicating that the classifier is struggling to make accurate predictions due to the noisy labels. The reduce in K implies that maybe model think that in a noisy dataset, closer neighbors might still be more reliable, and increasing k could mean that more noisy labels are included in the decision-making process, which could degrade the performance further. However the reduced k value does not compensate for the noise in this case (shown by the low accuracy), because KNN relies on the assumption that labels of nearby instances are likely to be similar, label noise directly undermines this assumption, leading to poor model performance.